

LANGUAGE TRANSLATION TOOL

Complete Project Task / Mini Project Report

Project Type: Web-based application

Purpose: Convert text from a selected source language into a selected target language using a translation API.

Prepared For	Project / Internship Task
Technology	HTML, CSS, JavaScript, Translation API
Main Features	Translation, Copy, Clear, Text-to-Speech

Submitted as a complete implementation-oriented task document.

1. Introduction

A Language Translation Tool is a software application that converts written text from one language into another. The application provides a simple user interface where a user enters text, selects the source language and target language, and requests a translation. The entered text is then sent to a translation service through an API, and the translated response is displayed clearly.

The project demonstrates how a web application can communicate with an external API, process a response, and present useful information to the user. Optional usability features such as copying the translated text and converting it into speech can make the tool more convenient.

2. Objectives

- Create a simple and user-friendly translation interface.
- Allow users to enter text and select source and target languages.
- Connect the application to a translation API.
- Send the input text to the API and receive the translated response.
- Display the translated result clearly.
- Provide copy and text-to-speech options for better usability.
- Handle empty input and API errors gracefully.

3. Functional Requirements

- Text input area for the original sentence or paragraph.
- Source-language selection.
- Target-language selection.
- Translate button to start the request.
- Output area for the translated text.
- Copy button to copy the result.
- Text-to-speech button to read the result aloud.
- Clear/reset option.
- Error message when input or API response is invalid.

4. System Design and Working

The application follows a simple request-response flow. The user interacts with the web page, JavaScript validates the input, the application sends a request to the translation API, and the returned translation is displayed in the output section.

Step	Operation
1	User enters text.
2	User selects source and target languages.
3	User clicks Translate.
4	JavaScript validates the input.
5	Request is sent to the translation API.
6	API processes the text and returns the translation.
7	Translated text is displayed.
8	User can copy or listen to the result.

Overall flow: User Input → Language Selection → Validation → Translation API → Response → Display → Copy / Text-to-Speech

5. User Interface

- Header: displays the application name.
- Input panel: contains the text area and source-language selector.
- Target selector: allows the user to choose the output language.
- Translate button: sends the request.
- Result panel: displays the translated text.
- Action buttons: Copy, Speak and Clear.

6. API Integration

A translation API such as Google Cloud Translation API or Microsoft Translator can be used. The application sends the text together with the source and target language codes. The service processes the request and returns the translated text in its response.

Important: An API key or authentication credential should never be hard-coded into a public client-side application. For a production application, keep the credential on a secure backend/server and call the translation service from there.

7. Implementation

Below is a complete front-end implementation structure. The interface is ready to connect to a translation API. The API call is represented through a backend endpoint such as `/api/translate`, which keeps the API credential away from the browser.

7.1 HTML - User Interface

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Language Translation Tool</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
    <h1>Language Translation Tool</h1>

    <textarea id="inputText"
      placeholder="Enter text to translate..."></textarea>

    <div class="row">
      <select id="sourceLang">
        <option value="en">English</option>
        <option value="te">Telugu</option>
        <option value="hi">Hindi</option>
      </select>

      <select id="targetLang">
        <option value="te">Telugu</option>
        <option value="en">English</option>
        <option value="hi">Hindi</option>
      </select>
    </div>

    <button onclick="translateText()">Translate</button>

    <textarea id="outputText"
      placeholder="Translated text will appear here..."
      readonly></textarea>

    <div class="actions">
      <button onclick="copyText()">Copy</button>
      <button onclick="speakText()">Speak</button>
      <button onclick="clearText()">Clear</button>
    </div>
  </div>
  <script src="script.js"></script>
</body>
</html>
```

7.2 CSS - Simple Styling

```
body {
  font-family: Arial, sans-serif;
  background: #f4f4f4;
  margin: 0;
  padding: 40px;
}

.container {
  max-width: 700px;
  margin: auto;
  background: white;
  padding: 25px;
  border-radius: 12px;
}
```

```
textarea, select, button {  
  width: 100%;  
  margin-top: 12px;  
  padding: 12px;  
  box-sizing: border-box;  
}  
  
.row, .actions {  
  display: flex;  
  gap: 10px;  
}  
  
button {  
  cursor: pointer;  
}
```

7.3 JavaScript - Translation and Usability Functions

```
async function translateText() {
  const text = document.getElementById("inputText").value.trim();
  const source = document.getElementById("sourceLang").value;
  const target = document.getElementById("targetLang").value;
  const output = document.getElementById("outputText");

  if (!text) {
    output.value = "Please enter text to translate.";
    return;
  }

  output.value = "Translating...";

  try {
    const response = await fetch("/api/translate", {
      method: "POST",
      headers: {"Content-Type": "application/json"},
      body: JSON.stringify({text, source, target})
    });

    if (!response.ok) throw new Error("Translation failed.");

    const data = await response.json();
    output.value = data.translation;
  } catch (error) {
    output.value = "Unable to translate. Please try again.";
  }
}

function copyText() {
  const output = document.getElementById("outputText");
  navigator.clipboard.writeText(output.value);
}

function speakText() {
  const text = document.getElementById("outputText").value;
  if (!text) return;

  const speech = new SpeechSynthesisUtterance(text);
  window.speechSynthesis.speak(speech);
}

function clearText() {
  document.getElementById("inputText").value = "";
  document.getElementById("outputText").value = "";
}
```

7.4 Backend/API Logic

The backend receives the text, source language and target language, then calls the selected translation service using the securely stored API credential. It returns a small JSON response such as { "translation": "..." } to the browser. The exact backend code depends on the chosen provider and programming language.

8. Testing

Test Case	Expected Result
Enter English text and select Telugu.	Telugu translation is displayed.
Leave input empty.	Validation message is displayed.
Click Copy.	Translated text is copied.

Click Speak.	Translated text is read aloud.
Click Clear.	Input and output are cleared.
API failure/network error.	User-friendly error message is shown.

9. Advantages

- Easy and quick translation of text.
- Simple interface suitable for beginners.
- Supports multiple languages through the API.
- Reduces manual translation effort.
- Copy and speech features improve usability.
- Can be extended into a mobile or desktop application.

10. Future Enhancements

- Add more languages.
- Add voice input using speech recognition.
- Add automatic language detection.
- Add translation history.
- Add a responsive mobile-first design.
- Add user authentication and saved translations.
- Add secure backend logging and API usage monitoring.

11. Conclusion

The Language Translation Tool is a practical application of web development and API integration. It accepts user text, allows language selection, sends the request to a translation service, and displays the translated result. Additional features such as copying and text-to-speech improve the overall user experience. The project also demonstrates important concepts including user-interface design, asynchronous API requests, error handling, and secure API integration.

12. Final Project Flow

Enter Text → Select Source Language → Select Target Language → Click Translate → API Request → Receive Translation → Display Result → Copy / Speak

This document covers the complete task requirements, working, design, implementation structure, testing, advantages, and future enhancements.